

PROMPT MESTRE — SISTEMA CEC ENGENHARIA E CAPACITAÇÃO

Cole este arquivo no Antigravity e instrua o agente a ler e executar

CONTEXTO DO PROJETO

Você está trabalhando no projeto **cursocec.com.br** — plataforma LMS da C&C Engenharia e Capacitação.

Stack atual (já funcionando em produção)

- **Frontend:** React + Vite + TypeScript — deploy na Hostinger via GitHub Actions
 - **Banco de dados:** Supabase Cloud (PostgreSQL) — projeto "C&C CURSO" (Supabase Cloud)
 - **Autenticação:** Supabase Auth
 - **Storage:** Supabase Storage
 - **Automações:** N8N + Evolution API (WhatsApp) — VPS Hostinger com EasyPanel
 - **IA do WhatsApp:** Maria Antônia — agente N8N com ChatGPT (GPT-4) que responde dúvidas e informa valores, carga horária e previsão de turmas buscando dados em tempo real da secretaria (Supabase)
 - **Pagamentos:** Asaas (a integrar — gateway BR com PIX, boleto, cartão)
 - **Domínio:** cursocec.com.br
-

CONFIGURAÇÃO DO BANCO DE DADOS

Supabase Cloud — C&C CURSO (banco principal)

URL: <https://sedwwagwrkfaiptoemsp.supabase.co>

Projeto: C&C CURSO

Status:  Migrado para Cloud — 28 tabelas intactas

Variáveis de ambiente (atualizar na Hostinger após migração completa):

VITE_SUPABASE_URL=https://sedwwagwrkfaiptoemsp.supabase.co

VITE_SUPABASE_ANON_KEY=[chave-anonima-do-projeto-cec-curso]

⚠ O SITE CEC (banco antigo) não é mais usado. Toda a operação aponta para C&C CURSO.

⚠ REGRAS ABSOLUTAS — LER ANTES DE QUALQUER AÇÃO

REGRA 1 — Layout e cores: NUNCA alterar

O site possui identidade visual definida (cores, fontes, espaçamentos, componentes). Todos os módulos novos (secretaria, aluno, instrutor, checkout) DEVEM seguir exatamente o mesmo padrão visual já existente no projeto.

- Reutilize os componentes, classes Tailwind e variáveis CSS já existentes
- Não instale novas bibliotecas de UI sem confirmação
- Não crie novos temas, paletas ou sistemas de design
- Copie o padrão visual de componentes existentes (botões, cards, tabelas, formulários)

REGRA 2 — Página principal: não alterar lógica e rotas

A página principal está funcionando em produção. O webdesign (visual) ainda pode ser ajustado. e NÃO devem ser alteradas de nenhuma forma.

- Não editar: src/pages/public/ (nem um caractere)
- Não editar: rotas públicas já existentes
- Não editar: componentes do site público
- EXCEÇÃO: a página principal LÊ dados da secretaria (Supabase) para exibir informações de cursos, valores, carga horária e previsão de turmas. Isso já funciona via queries ao Supabase. Se a secretaria atualizar esses dados, a página principal reflete automaticamente. Não alterar a lógica de leitura.

REGRA 3 — Confirmar antes de criar tabelas

Antes de rodar qualquer SQL no Supabase, confirme com o usuário. Os SQLs deste arquivo são sugestões — o usuário os revisa e executa manualmente.

REGRA 4 — Aulas práticas são nos finais de semana

⚠️ **NOTA IMPORTANTE DE NEGÓCIO:** As aulas práticas (presenciais) da C&C são realizadas exclusivamente nos **FINAIS DE SEMANA** (sábados e domingos).

- Ao criar o calendário de turmas, o seletor de data deve bloquear dias úteis
 - Sugestão automática de datas deve oferecer apenas sábados e domingos
 - Labels no sistema devem refletir: "Aula prática — Final de semana"
 - O agendamento de turmas na secretaria deve validar: somente sábado ou domingo
-

O QUE NÃO MEXER

- Site público já está pronto. Não alterar páginas de home, sobre, contato, serviços
- Não alterar layout, design, cores ou componentes visuais existentes
- Não alterar rotas públicas existentes

O QUE CONSTRUIR

Todo o sistema interno — mantendo a identidade visual existente:

- `/secretaria` — painel admin/secretaria
 - `/aluno` — portal do aluno (LMS)
 - `/instrutor` — portal do instrutor
 - Integração com Asaas (pagamentos)
 - Integração com N8N / Maria Antônia (WhatsApp + IA)
-

MARIA ANTÔNIA — AGENTE DE WHATSAPP COM IA

Maria Antônia é o nome do agente de atendimento automatizado da C&C via WhatsApp, construído no N8N com integração ChatGPT (GPT-4) e Evolution API.

O que Maria Antônia faz

- Responde dúvidas sobre os cursos (CD-MC, CD-CL, CD-TO e futuros)
- Informa valores, condições de pagamento (PIX, boleto, parcelamento)
- Informa carga horária de cada curso
- Informa previsão de abertura de novas turmas
- Informa datas das próximas aulas práticas (finais de semana)

- Encaminha interessados para o link de matrícula
- Escala para atendente humano quando necessário (opção no menu)

De onde Maria Antônia busca os dados

TODOS os dados que Maria Antônia usa vêm diretamente do Supabase (secretaria). Isso garante que qualquer atualização feita pela secretaria reflete imediatamente nas respostas do WhatsApp e na página principal do site.

Informação	Tabela Supabase	Campo
Nome do curso	lms_courses	title
Valor do curso	lms_courses	price
Carga horária total	lms_courses	carga_horaria_total
Carga horária online	lms_courses	carga_horaria_online
Carga horária presencial	lms_courses	carga_horaria_presencial
Próximas turmas	upcoming_classes	date, start_time, capacity, enrolled_count
Vagas disponíveis	upcoming_classes	capacity - enrolled_count
Norma técnica	lms_courses	norma_tecnica
Modalidade	lms_courses	modality

Endpoint da Secretaria para Maria Antônia

A secretaria expõe uma Supabase Edge Function ou RPC que o N8N consulta:

-- View pública que Maria Antônia e o site principal consomem

```
CREATE OR REPLACE VIEW vw_course_info AS
```

```
SELECT
```

```
  c.id,
```

```
  c.title,
```

```
  c.price,
```

c.original_price,

c.modality,

c.carga_horaria_total,

c.carga_horaria_online,

c.carga_horaria_presencial,

c.norma_tecnica,

c.level,

c.status,

-- Próxima turma disponível

(SELECT date FROM upcoming_classes uc

WHERE uc.course_id = c.id

AND uc.date >= CURRENT_DATE

AND uc.status = 'scheduled'

AND (uc.capacity - uc.enrolled_count) > 0

ORDER BY uc.date ASC

LIMIT 1) AS proxima_turma_data,

-- Vagas na próxima turma

(SELECT (capacity - enrolled_count) FROM upcoming_classes uc

WHERE uc.course_id = c.id

AND uc.date >= CURRENT_DATE

AND uc.status = 'scheduled'

```
ORDER BY uc.date ASC

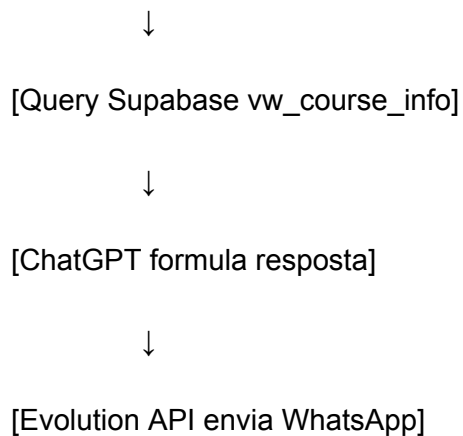
LIMIT 1) AS proxima_turma_vagas

FROM lms_courses c

WHERE c.status = 'published';
```

Fluxo N8N — Maria Antônia buscando dados

[WhatsApp mensagem] → [N8N webhook] → [ChatGPT com contexto]



O que a Secretaria precisa manter atualizado para Maria Antônia funcionar

- Valores dos cursos (lms_courses.price) — Maria Antônia informa o preço correto
- Carga horária (lms_courses.carga_horaria_total) — informada nas respostas
- Turmas abertas (upcoming_classes) — Maria Antônia informa datas e vagas
- Status de publicação (lms_courses.status = 'published') — só cursos publicados aparecem

Integrações a criar no N8N para Maria Antônia

Webhook / Trigger	Ação	Resultado
Aluno pergunta sobre curso	Consulta vw_course_info	Resposta com valor + carga horária + turma
Aluno pergunta sobre turmas	Consulta upcoming_classes	Datas disponíveis + vagas
Nova turma criada na secretaria	Trigger automático	Maria Antônia é notificada para informar leads

Webhook / Trigger	Ação	Resultado
Turma com vagas < 20%	Alert automático	Maria Antônia prioriza essa turma nas respostas
Aluno pede link de matrícula	Retorna link do checkout	/checkout/:course_id
Pagamento confirmado	Asaas webhook → N8N	Boas-vindas + acesso ao portal
24h antes da aula prática	Cron N8N	Lembrete com endereço e horário
Falta registrada	Registro de presença	Aviso ao aluno via WhatsApp
Certificado disponível	Conclusão do curso	Link para download

ORGANIZAÇÃO DAS AULAS PRÁTICAS (PRESENCIAIS)

Regra de negócio — finais de semana apenas

As aulas práticas da C&C são realizadas exclusivamente nos finais de semana. Isso é uma regra de negócio fixa — não negociável.

Como organizar na secretaria

Criação de turma:

- Secretaria acessa: Secretaria → Turmas → Nova Turma
- Seleciona o curso (ex: CD-MC)
- Seleciona os módulos que serão presenciais nessa turma
- Define as datas (SOMENTE sábados e domingos — calendário bloqueia dias úteis)
- Define horário (ex: 08h00 às 17h00)
- Define endereço (sede C&C ou local externo)
- Define capacidade máxima de alunos
- Define instrutor responsável
- Sistema calcula automaticamente: quantas aulas práticas, carga horária presencial total

Exemplo de organização de turma CD-MC:

Turma CD-MC — Junho/Julho 2026

Aula Prática 1 — Sábado 14/06/2026 — 08h às 17h — Sede C&C

Aula Prática 2 — Domingo 15/06/2026 — 08h às 12h — Sede C&C

Aula Prática 3 — Sábado 21/06/2026 — 08h às 17h — Campo (endereço definido)

Prova Final — Sábado 28/06/2026 — 08h às 12h — Sede C&C

Total presencial: 36 horas

Validações no formulário de nova turma:

// Validar que a data selecionada é sábado (6) ou domingo (0)

```
function isWeekend(date: Date): boolean {
```

```
    const day = date.getDay();
```

```
    return day === 0 || day === 6;
```

```
}
```

// No DatePicker — desabilitar dias úteis

```
disabledDays={(date) => !isWeekend(date)}
```

Campos da tabela upcoming_classes a verificar/adicionar:

```
ALTER TABLE upcoming_classes
```

```
ADD COLUMN IF NOT EXISTS address TEXT,
```

```
ADD COLUMN IF NOT EXISTS address_complement TEXT,
```

```
ADD COLUMN IF NOT EXISTS start_time TIME,
```

```
ADD COLUMN IF NOT EXISTS end_time TIME,
```

```
ADD COLUMN IF NOT EXISTS capacity INT DEFAULT 20,
```

```
ADD COLUMN IF NOT EXISTS enrolled_count INT DEFAULT 0,
```

```
ADD COLUMN IF NOT EXISTS whatsapp_group_url TEXT,
```

ADD COLUMN IF NOT EXISTS lesson_type VARCHAR(20) DEFAULT 'pratica'

CHECK (lesson_type IN ('pratica','prova','reposicao')),

ADD COLUMN IF NOT EXISTS notes TEXT,

ADD COLUMN IF NOT EXISTS status VARCHAR(20) DEFAULT 'scheduled'

CHECK (status IN ('scheduled','completed','cancelled'));

No portal do aluno — card de aula prática:

 Aula Prática — Final de semana

Sábado, 14 de junho de 2026

08h00 às 17h00

Sede C&C — Endereço completo aqui

Instrutor: [Nome do instrutor]

Vagas restantes: 8 de 20

[ Confirmar presença] [ Ver no Maps] [ Grupo da turma]

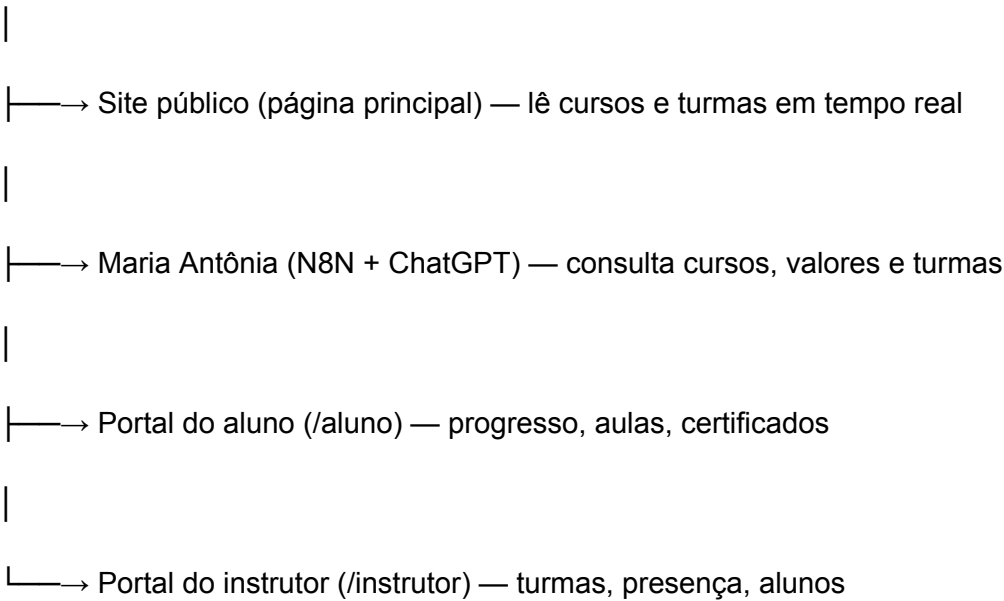
Notificações automáticas via Maria Antônia / N8N:

- Quando turma é criada → Maria Antônia pode informar leads sobre datas
 - 72h antes → WhatsApp com data, horário e endereço
 - 24h antes → Lembrete final com checklist (o que levar, EPI, documentos)
 - No dia → WhatsApp às 7h30 com "Bom dia! Sua aula prática começa às 8h..."
 - Após a aula → Confirmação de presença registrada + próximos passos
-

FONTE ÚNICA DE DADOS — SECRETARIA → SITE → MARIA ANTÔNIA

 **PRINCÍPIO CENTRAL DO SISTEMA:** A secretaria é a ÚNICA fonte de verdade. Toda informação que aparece no site público e nas respostas do WhatsApp (Maria Antônia) vem do Supabase.

SECRETARIA (Supabase)



O que a secretaria atualiza e onde reflete:

Ação na secretaria	Onde reflete automaticamente
Publicar novo curso	Site público + Maria Antônia
Atualizar preço	Site público + Maria Antônia
Abrir nova turma	Site público + Maria Antônia + Portal aluno
Cancelar turma	Site público + Maria Antônia + Notificação alunos
Atualizar carga horária	Site público + Maria Antônia
Fechar vagas	Maria Antônia para de oferecer a turma

TABELAS JÁ EXISTENTES NO SUPABASE

As seguintes tabelas já existem. Use-as — NÃO recriar:

students	— alunos cadastrados
lms_courses	— cursos da plataforma
lms_modules	— módulos de cada curso
lms_lessons	— lições de cada módulo
lms_student_progress	— progresso do aluno por curso
lms_quiz_results	— resultados de quizzes
lms_quizzes	— quizzes por lição
lms_time_logs	— tempo de estudo por lição
lms_certificate_configs	— configurações de certificado
enrollments	— matrículas dos alunos
upcoming_classes	— turmas/aulas presenciais agendadas
attendance_records	— registro de presença em aulas presenciais
financial_records	— registros financeiros
class_instructors	— instrutores por turma
users	— usuários do sistema
system_settings	— configurações gerais do sistema
testimonials	— depoimentos
site_content	— conteúdo editável do site
complaints	— reclamações/tickets

TABELAS A CRIAR / ALTERAR NO SUPABASE

⚠ Execute estes SQLs no Supabase SQL Editor ANTES de rodar o Antigravity. Cada SQL usa IF NOT EXISTS — seguro para rodar mesmo que parcialmente já exista.

1. View pública para Site + Maria Antônia

```
CREATE OR REPLACE VIEW vw_course_info AS
```

```
SELECT
```

```
  c.id, c.title, c.price, c.original_price, c.modality,
```

```
  c.carga_horaria_total, c.carga_horaria_online, c.carga_horaria_presencial,
```

```
  c.norma_tecnica, c.level, c.status,
```

```
(SELECT date FROM upcoming_classes uc
```

```
  WHERE uc.course_id = c.id AND uc.date >= CURRENT_DATE
```

```
  AND uc.status = 'scheduled' AND (uc.capacity - uc.enrolled_count) > 0
```

```
  ORDER BY uc.date ASC LIMIT 1) AS proxima_turma_data,
```

```
(SELECT (capacity - enrolled_count) FROM upcoming_classes uc
```

```
  WHERE uc.course_id = c.id AND uc.date >= CURRENT_DATE
```

```
  AND uc.status = 'scheduled' ORDER BY uc.date ASC LIMIT 1) AS proxima_turma_vagas
```

```
FROM lms_courses c WHERE c.status = 'published';
```

2. Colunas em lms_lessons (tipos de aula)

```
ALTER TABLE lms_lessons
```

```
  ADD COLUMN IF NOT EXISTS type VARCHAR(20) DEFAULT 'video'
```

```
  CHECK (type IN ('video', 'pdf', 'texto', 'quiz', 'presencial', 'tarefa', 'zoom'));
```

```
ADD COLUMN IF NOT EXISTS upcoming_class_id UUID REFERENCES  
upcoming_classes(id),
```

```
ADD COLUMN IF NOT EXISTS video_url TEXT,
```

```
ADD COLUMN IF NOT EXISTS pdf_url TEXT,
```

```
ADD COLUMN IF NOT EXISTS content_html TEXT,
```

```
ADD COLUMN IF NOT EXISTS duration_minutes INT,
```

```
ADD COLUMN IF NOT EXISTS is_preview BOOLEAN DEFAULT false,
```

```
ADD COLUMN IF NOT EXISTS unlock_after_days INT DEFAULT 0;
```

3. Colunas em lms_courses (modalidade + configurações)

```
ALTER TABLE lms_courses
```

```
ADD COLUMN IF NOT EXISTS modality VARCHAR(20) DEFAULT 'hibrido'
```

```
  CHECK (modality IN ('online','presencial','hibrido')),
```

```
ADD COLUMN IF NOT EXISTS min_attendance_percent INT DEFAULT 75,
```

```
ADD COLUMN IF NOT EXISTS min_quiz_score INT DEFAULT 70,
```

```
ADD COLUMN IF NOT EXISTS carga_horaria_total INT,
```

```
ADD COLUMN IF NOT EXISTS carga_horaria_online INT,
```

```
ADD COLUMN IF NOT EXISTS carga_horaria_presencial INT,
```

```
ADD COLUMN IF NOT EXISTS norma_tecnica VARCHAR(100),
```

```
ADD COLUMN IF NOT EXISTS docs_required JSONB DEFAULT '[]',
```

```
ADD COLUMN IF NOT EXISTS original_price DECIMAL(10,2),
```

```
ADD COLUMN IF NOT EXISTS level VARCHAR(20) DEFAULT 'iniciante'
```

```
  CHECK (level IN ('iniciante','intermediario','avancado')),
```

```
ADD COLUMN IF NOT EXISTS preview_video_url TEXT,  
  
ADD COLUMN IF NOT EXISTS banner_image_url TEXT,  
  
ADD COLUMN IF NOT EXISTS outcomes JSONB DEFAULT '[]',  
  
ADD COLUMN IF NOT EXISTS requirements JSONB DEFAULT '[]',  
  
ADD COLUMN IF NOT EXISTS certificate_requires_attendance BOOLEAN DEFAULT true,  
  
ADD COLUMN IF NOT EXISTS certificate_requires_quiz BOOLEAN DEFAULT true;
```

4. Colunas em lms_modules

```
ALTER TABLE lms_modules
```

```
ADD COLUMN IF NOT EXISTS modality VARCHAR(20) DEFAULT 'online'  
  
CHECK (modality IN ('online','presencial'));
```

5. Colunas em upcoming_classes (aulas práticas de fim de semana)

```
ALTER TABLE upcoming_classes
```

```
ADD COLUMN IF NOT EXISTS address TEXT,  
  
ADD COLUMN IF NOT EXISTS address_complement TEXT,  
  
ADD COLUMN IF NOT EXISTS start_time TIME,  
  
ADD COLUMN IF NOT EXISTS end_time TIME,  
  
ADD COLUMN IF NOT EXISTS capacity INT DEFAULT 20,  
  
ADD COLUMN IF NOT EXISTS enrolled_count INT DEFAULT 0,  
  
ADD COLUMN IF NOT EXISTS whatsapp_group_url TEXT,  
  
ADD COLUMN IF NOT EXISTS lesson_type VARCHAR(20) DEFAULT 'pratica'  
  
CHECK (lesson_type IN ('pratica','prova','reposicao')),  
  
ADD COLUMN IF NOT EXISTS notes TEXT,
```

```
ADD COLUMN IF NOT EXISTS status VARCHAR(20) DEFAULT 'scheduled'
```

```
CHECK (status IN ('scheduled','completed','cancelled'));
```

6. Tabela orders (pagamentos Asaas)

```
CREATE TABLE IF NOT EXISTS orders (
```

```
id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
```

```
student_id UUID REFERENCES students(id),
```

```
course_id UUID REFERENCES lms_courses(id),
```

```
amount DECIMAL(10,2) NOT NULL,
```

```
payment_method VARCHAR(20) CHECK (payment_method IN  
( 'pix','boleto','credit_card','manual')),
```

```
installments INT DEFAULT 1,
```

```
status VARCHAR(20) DEFAULT 'pending'
```

```
CHECK (status IN ('pending','paid','failed','refunded','overdue')),
```

```
asaas_payment_id VARCHAR(100),
```

```
asaas_customer_id VARCHAR(100),
```

```
pix_qr_code TEXT,
```

```
pix_copy_paste TEXT,
```

```
boleto_url TEXT,
```

```
boleto_barcode TEXT,
```

```
paid_at TIMESTAMPTZ,
```

```
due_date DATE,
```

```
created_at TIMESTAMPTZ DEFAULT NOW()
```

);

7. Tabela asaas_customers

```
CREATE TABLE IF NOT EXISTS asaas_customers (  
  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  
  student_id UUID REFERENCES students(id) UNIQUE,  
  
  asaas_customer_id VARCHAR(100) UNIQUE NOT NULL,  
  
  created_at TIMESTAMPTZ DEFAULT NOW()  
  
);
```

8. Tabela asaas_webhooks

```
CREATE TABLE IF NOT EXISTS asaas_webhooks (  
  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  
  event VARCHAR(50),  
  
  payment_id VARCHAR(100),  
  
  payload JSONB,  
  
  processed BOOLEAN DEFAULT false,  
  
  created_at TIMESTAMPTZ DEFAULT NOW()  
  
);
```

9. Tabela coupons

```
CREATE TABLE IF NOT EXISTS coupons (  
  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  
  code VARCHAR(50) UNIQUE NOT NULL,  
  
  type VARCHAR(10) CHECK (type IN ('percent','fixed')),
```

```
value DECIMAL(10,2) NOT NULL,  
  
max_uses INT,  
  
used_count INT DEFAULT 0,  
  
course_id UUID REFERENCES lms_courses(id),  
  
expires_at TIMESTAMPTZ,  
  
active BOOLEAN DEFAULT true,  
  
created_at TIMESTAMPTZ DEFAULT NOW()  
  
);
```

10. Tabela course_reviews

```
CREATE TABLE IF NOT EXISTS course_reviews (  
  
id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  
course_id UUID REFERENCES lms_courses(id),  
  
student_id UUID REFERENCES students(id),  
  
rating INT CHECK (rating BETWEEN 1 AND 5),  
  
comment TEXT,  
  
created_at TIMESTAMPTZ DEFAULT NOW(),  
  
UNIQUE(course_id, student_id)  
  
);
```

11. Tabela notifications

```
CREATE TABLE IF NOT EXISTS notifications (  
  
id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  
user_id UUID REFERENCES users(id),
```

```
type VARCHAR(50),  
  
title VARCHAR(200),  
  
body TEXT,  
  
read BOOLEAN DEFAULT false,  
  
link VARCHAR(500),  
  
created_at TIMESTAMPTZ DEFAULT NOW()  
  
);
```

12. Tabela student_documents

```
CREATE TABLE IF NOT EXISTS student_documents (  
  
id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  
student_id UUID REFERENCES students(id),  
  
course_id UUID REFERENCES lms_courses(id),  
  
doc_type VARCHAR(100),  
  
file_url TEXT,  
  
status VARCHAR(20) DEFAULT 'pending'  
  
CHECK (status IN ('pending','approved','rejected')),  
  
reviewed_by UUID REFERENCES users(id),  
  
created_at TIMESTAMPTZ DEFAULT NOW()  
  
);
```

13. Tabela task_submissions

```
CREATE TABLE IF NOT EXISTS task_submissions (  
  
id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
```

```
student_id UUID REFERENCES students(id),

lesson_id UUID REFERENCES lms_lessons(id),

file_url TEXT,

status VARCHAR(20) DEFAULT 'submitted'

CHECK (status IN ('submitted','approved','revision')),

instructor_feedback TEXT,

submitted_at TIMESTAMPTZ DEFAULT NOW(),

reviewed_at TIMESTAMPTZ

);
```

ESTRUTURA DE PASTAS DO PROJETO

Organize o projeto React desta forma (crie as pastas que não existirem):

src/

├── pages/

| ├── public/ ←  NÃO TOCAR — site já pronto

| ├── secretaria/ ← CONSTRUIR

| | ├── Dashboard.tsx

| | ├── Alunos.tsx

| | ├── AlunoDetalhe.tsx

| | ├── Cursos.tsx

| | ├── CursoEditor.tsx

- | | | — CurriculoEditor.tsx
- | | | — Turmas.tsx ← aulas práticas (finais de semana)
- | | | — TurmaDetalhe.tsx
- | | | — Instrutores.tsx
- | | | — Financeiro.tsx
- | | | — Relatorios.tsx
- | | | — Configuracoes.tsx
- | | — aluno/ ← CONSTRUIR
- | | | — Dashboard.tsx
- | | | — MeusCursos.tsx
- | | | — CursoPlayer.tsx
- | | | — Certificados.tsx
- | | | — Historico.tsx
- | | | — Perfil.tsx
- | | — instrutor/ ← CONSTRUIR
- | | | — Dashboard.tsx
- | | | — MinhasTurmas.tsx
- | | | — Presenca.tsx
- | | | — MeusCursos.tsx
- | | | — Alunos.tsx
- | — components/
- | | — secretaria/

- | | └─ KPICard.tsx
- | | └─ RevenueChart.tsx
- | | └─ EnrollmentChart.tsx
- | | └─ CourseForm.tsx
- | | └─ LessonEditor.tsx
- | | └─ TurmaForm.tsx ← com validação de fim de semana
- | | └─ StudentTable.tsx
- | └─ aluno/
- | | └─ CourseCard.tsx
- | | └─ ProgressBar.tsx
- | | └─ LessonSidebar.tsx
- | | └─ VideoPlayer.tsx
- | | └─ QuizForm.tsx
- | | └─ PresencialCard.tsx ← card de aula prática
- | | └─ CertificateCard.tsx
- | └─ checkout/
- | | └─ CheckoutForm.tsx
- | | └─ PixDisplay.tsx
- | | └─ BoletoDisplay.tsx
- | | └─ CreditCardForm.tsx
- | └─ shared/

- | |— Sidebar.tsx
- | |— Header.tsx
- | |— NotificationBell.tsx
- | |— Badge.tsx
- | |— Modal.tsx
- |— hooks/
 - | |— useSupabase.ts
 - | |— useAuth.ts
 - | |— useCourses.ts
 - | |— useEnrollments.ts
 - | |— useProgress.ts
 - | |— useAsaas.ts
- |— services/
 - | |— supabase.ts ← cliente Supabase já configurado
 - | |— asaas.ts ← chamadas à API Asaas
 - | |— n8n.ts ← webhooks para N8N / Maria Antônia
- |— types/
 - | |— course.ts
 - | |— student.ts
 - | |— enrollment.ts
 - | |— order.ts
 - | |— lesson.ts

TIPOS TYPESCRIPT

course.ts

```
export type CourseModality = 'online' | 'presencial' | 'hibrido';
```

```
export type CourseLevel = 'iniciante' | 'intermediario' | 'avancado';
```

```
export type CourseStatus = 'draft' | 'published' | 'archived';
```

```
export interface Course {
```

```
  id: string;
```

```
  title: string;
```

```
  slug: string;
```

```
  description: string;
```

```
  modality: CourseModality;
```

```
  level: CourseLevel;
```

```
  status: CourseStatus;
```

```
  price: number;
```

```
  original_price?: number;
```

```
  thumbnail_url?: string;
```

```
  banner_image_url?: string;
```

```
  preview_video_url?: string;
```

```
  carga_horaria_total?: number;
```

```
  carga_horaria_online?: number;
```

```
carga_horaria_presencial?: number;

norma_tecnica?: string;

min_attendance_percent: number;

min_quiz_score: number;

docs_required: string[];

outcomes: string[];

requirements: string[];

certificate_requires_attendance: boolean;

certificate_requires_quiz: boolean;

created_at: string;

}
```

lesson.ts

```
export type LessonType = 'video' | 'pdf' | 'texto' | 'quiz' | 'presencial' | 'tarefa' | 'zoom';

export interface Lesson {

  id: string;

  module_id: string;

  title: string;

  type: LessonType;

  order_index: number;

  video_url?: string;

  pdf_url?: string;
```

```
content_html?: string;

duration_minutes?: number;

is_preview: boolean;

unlock_after_days: number;

upcoming_class_id?: string;

upcoming_class?: UpcomingClass;
}

export interface UpcomingClass {

  id: string;

  course_id: string;

  date: string;      // sempre sábado ou domingo

  start_time: string;

  end_time: string;

  address: string;

  address_complement?: string;

  lesson_type: 'pratica' | 'prova' | 'reposicao';

  capacity: number;

  enrolled_count: number;

  whatsapp_group_url?: string;

  notes?: string;

  status: 'scheduled' | 'completed' | 'cancelled';
}
```

order.ts

```
export type PaymentMethod = 'pix' | 'boleto' | 'credit_card' | 'manual';

export type OrderStatus = 'pending' | 'paid' | 'failed' | 'refunded' | 'overdue';

export interface Order {

  id: string;

  student_id: string;

  course_id: string;

  amount: number;

  payment_method: PaymentMethod;

  installments: number;

  status: OrderStatus;

  asaas_payment_id?: string;

  pix_qr_code?: string;

  pix_copy_paste?: string;

  boleto_url?: string;

  boleto_barcode?: string;

  paid_at?: string;

  due_date?: string;

  created_at: string;

}
```

MÓDULOS A CONSTRUIR — ORDEM DE PRIORIDADE

SPRINT 1 — Dashboard Secretaria + Portal Aluno Básico

1. Dashboard Secretaria (</secretaria/dashboard>)

KPIs em tempo real via Supabase — usar os mesmos componentes visuais do site:

- Total de alunos ativos
- Receita do mês (SUM orders.amount WHERE status='paid')
- Matrículas do mês
- Turmas abertas nos próximos 30 dias (upcoming_classes WHERE date BETWEEN hoje E hoje+30)
- Próximas aulas práticas (finais de semana — lista dos próximos 30 dias)
- Gráfico de receita mensal — últimos 6 meses (Recharts AreaChart)
- Gráfico de matrículas por curso (Recharts HorizontalBarChart)
- Alertas: turmas com vagas < 20%, cobranças vencidas

2. Portal do Aluno — Meus Cursos (</aluno/meus-cursos>)

- Lista de cursos matriculados com % de progresso
- Para cada curso: badge de modalidade 🎬 Online | 📍 Presencial | 🔄 Híbrido
- Próxima aula prática (se houver): "Sábado, 14/06 às 08h — Sede C&C"
- Botão "Continuar" → vai para o player na última lição

3. Player de Curso (</aluno/curso/:id>)

Sidebar com currículo — ícone por tipo:

- 🎬 video | 📄 pdf | 📝 texto | 📋 quiz | 📍 presencial | 📅 tarefa

Área principal por tipo:

- **video** → react-player
- **pdf** → iframe
- **texto** → HTML sanitizado
- **quiz** → QuizForm
- **presencial** → PresencialCard (ver detalhes abaixo)
- **tarefa** → upload + instruções

Componente PresencialCard — aulas práticas:

Aula Prática — Final de semana

[Sábado / Domingo], DD de Mês de AAAA

HH:mm às HH:mm

[Endereço completo]

Instrutor: [Nome]

Vagas: X de Y disponíveis

[ Confirmar presença] [ Ver no Maps] [ Grupo da turma]

Nota: Traga seus EPIs e documentos necessários para a aula prática.

SPRINT 2 — Checkout + Asaas

Variáveis de ambiente:

VITE_ASAAS_API_KEY=sua_chave

VITE_ASAAS_BASE_URL=https://sandbox.asaas.com/api/v3

Serviço Asaas ([src/services/asaas.ts](#)):

```
const BASE = import.meta.env.VITE_ASAAS_BASE_URL;
```

```
const KEY = import.meta.env.VITE_ASAAS_API_KEY;
```

```
const headers = { 'Content-Type': 'application/json', 'access_token': KEY };
```

```
export const asaas = {
```

```
  createCustomer: (student) =>
```

```
    fetch(`${BASE}/customers`, { method: 'POST', headers, body: JSON.stringify({
```

```
      name: student.name, cpfCnpj: student.cpf,
```

```
email: student.email, mobilePhone: student.phone

}})).then(r => r.json()),

createPayment: (data) =>

fetch(`${BASE}/payments`, { method: 'POST', headers,

body: JSON.stringify(data) }).then(r => r.json()),

getPixQrCode: (paymentId) =>

fetch(`${BASE}/payments/${paymentId}/pixQrCode`, { headers }).then(r => r.json()),

};
```

SPRINT 3 — Portal do Instrutor

Dashboard do Instrutor:

- Minhas turmas (upcoming_classes WHERE instructor_id = user.id)
- Próxima aula prática (data, horário, endereço)
- Total de alunos nas minhas turmas

Registro de Presença (/instrutor/presenca/:turma_id):

- Lista de alunos da turma
 - Marcar: Presente  / Ausente  / Justificado 
 - Salvar → INSERT em attendance_records
 - N8N dispara WhatsApp para ausentes automaticamente via Maria Antônia
-

SPRINT 4 — Analytics + Financeiro Secretaria

Relatórios:

- Ranking cursos por receita e matrículas
- Taxa de conclusão por curso
- Alunos travados (progresso < 20% após 15 dias)
- Breakdown PIX vs Boleto vs Cartão
- Turmas com mais/menos adesão

- Previsão de receita (turmas abertas × preço × taxa histórica)

INTEGRAÇÕES N8N — RESUMO COMPLETO

Maria Antônia (atendimento WhatsApp + IA)

Trigger	Consulta Supabase	Resposta
Pergunta sobre curso	vw_course_info	Valor, carga horária, modalidade
Pergunta sobre turmas	upcoming_classes (fins de semana)	Datas, vagas, endereço
Pede matrícula	lms_courses.status = published	Link /checkout/:id
Pergunta sobre certificado	lms_courses (requisitos)	Frequência e nota mínimas

Automações de turmas presenciais

Trigger	Ação
Nova turma criada (fim de semana)	Alerta para coordenação + Maria Antônia atualizada
Turma com vagas < 20%	Alerta secretaria + Maria Antônia prioriza essa turma
72h antes da aula prática	WhatsApp com data, horário, endereço
24h antes da aula prática	Lembrete + checklist (EPIs, documentos)
No dia às 07h30	"Bom dia! Sua aula começa às 8h. Endereço: [X]"
Falta registrada	Aviso ao aluno + registra em attendance_records
Turma concluída	Verifica se aluno pode receber certificado

Automações de pagamento (Asaas → N8N)

Webhook Asaas	Ação N8N
PAYMENT_CONFIRMED	UPDATE orders + CREATE enrollment + WhatsApp boas-vindas
PAYMENT_RECEIVED	Idem (para boleto)
PAYMENT_OVERDUE	Alerta secretaria + WhatsApp cobrança para aluno
PAYMENT_REFUNDED	UPDATE orders status=refunded + notifica aluno

VARIÁVEIS DE AMBIENTE (.env)

VITE_SUPABASE_URL=https://sedwwagwrkfaiptoemsp.supabase.co

VITE_SUPABASE_ANON_KEY=[chave_anonima]

VITE_ASAAS_API_KEY=[chave_asaas]

VITE_ASAAS_BASE_URL=https://sandbox.asaas.com/api/v3

VITE_N8N_BASE_URL=https://[seu-n8n].com

VITE_EVOLUTION_API_URL=https://[sua-vps].com

INSTRUÇÕES PARA O ANTIGRAVITY

1. Leia este arquivo COMPLETO antes de qualquer ação
2. Respeite as 4 REGRAS ABSOLUTAS no topo do documento
3. Comece pela Sprint 1 — Dashboard Secretaria
4. NUNCA altere arquivos em src/pages/public/
5. SEMPRE use os componentes e estilos visuais já existentes no projeto
6. SEMPRE valide datas presenciais: apenas sábados e domingos
7. Instale apenas: recharts, react-player (confirmar antes de outros)

8. Ao finalizar cada sprint: liste o que foi criado e o que precisa ajuste manual
9. Antes de rodar SQL no Supabase: mostre o SQL e aguarde confirmação do usuário

Documento gerado por AgênciaDeIA | Piticalyn | cursocec.com.br | Maio 2026

CMS — GERENCIAMENTO DE CONTEÚDO DA PÁGINA PRINCIPAL

O que é

O cliente (C&C) acessa a secretaria e gerencia visualmente o conteúdo da página principal sem precisar de desenvolvedor. Qualquer alteração reflete imediatamente no site público.

Onde fica no sistema

/secretaria/conteudo-do-site

Seção dentro da secretaria chamada "Conteúdo do Site" ou "Webdesign".

O que o cliente pode gerenciar

Seção do site	O que pode alterar	Tabela Supabase
Banner / Hero	Imagem de fundo, título, subtítulo, texto do botão CTA	site_content
Vídeo principal	Upload ou URL do vídeo da home	site_content
Seção "Sobre"	Texto institucional, imagem	site_content
Cards de cursos	Automático — vem de lms_courses (publicados)	lms_courses
Próximas turmas	Automático — vem de upcoming_classes	upcoming_classes
Depoimentos	Adicionar, editar, remover, reordenar	testimonials

Seção do site	O que pode alterar	Tabela Supabase
Galeria de fotos	Upload de imagens	site_content (JSONB)
Redes sociais	Links Instagram, Facebook, YouTube, WhatsApp	system_settings
Informações de contato	Telefone, e-mail, endereço	system_settings
Logo	Upload da logo principal e versão escura	system_settings
Favicon	Upload do ícone do site	system_settings

Telas a construir em /secretaria/conteudo-do-site

1. Editor do Hero/Banner

- Preview em tempo real do como vai ficar
- Upload de imagem (Supabase Storage bucket: site-assets)
- Campo de URL do vídeo de fundo (YouTube embed ou upload direto)
- Campos de texto: título principal, subtítulo, texto do botão
- Botão "Salvar" → UPDATE site_content

2. Editor de Depoimentos

- Lista de depoimentos com foto, nome, cargo, empresa, texto
- Adicionar novo depoimento
- Upload de foto do cliente (Supabase Storage)
- Reordenar por drag & drop (campo order_index)
- Ativar/desativar depoimento sem excluir
- Tabela: testimonials

3. Editor de Galeria

- Upload múltiplo de imagens
- Reordenar por drag & drop
- Excluir imagem
- Armazenado em site_content como array JSONB de URLs

4. Configurações Gerais

- Logo principal + logo versão clara
- Favicon

- Links de redes sociais
- Telefone principal (usado pelo WhatsApp flutuante e Maria Antônia)
- E-mail de contato
- Endereço completo
- Tabela: system_settings (chave → valor)

Estrutura da tabela site_content

-- Verificar se existe, adicionar colunas se necessário

```
ALTER TABLE site_content
```

```
ADD COLUMN IF NOT EXISTS section VARCHAR(100),
```

```
ADD COLUMN IF NOT EXISTS key VARCHAR(100),
```

```
ADD COLUMN IF NOT EXISTS value_text TEXT,
```

```
ADD COLUMN IF NOT EXISTS value_json JSONB,
```

```
ADD COLUMN IF NOT EXISTS value_url TEXT,
```

```
ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ DEFAULT NOW();
```

-- Exemplos de registros

```
-- section='hero', key='titulo', value_text='C&C Engenharia e Capacitação'
```

```
-- section='hero', key='video_url', value_url='https://...'
```

```
-- section='hero', key='banner_image', value_url='https://storage.supabase...'
```

```
-- section='galeria', key='fotos', value_json='["url1","url2","url3"]'
```

Upload de arquivos — Supabase Storage

```
// src/services/storage.ts
```

```
import { supabase } from './supabase';
```

```
export async function uploadSiteAsset(file: File, path: string): Promise<string> {
```

```
  const { data, error } = await supabase.storage
```

```

    .from('site-assets')

    .upload(path, file, { upsert: true });

    if (error) throw error;

    const { data: { publicUrl } } = supabase.storage

    .from('site-assets')

    .getPublicUrl(data.path);

    return publicUrl;

}

// Uso no CMS:

// const url = await uploadSiteAsset(file, `hero/banner-${Date.now()}.jpg`);

// await updateSiteContent('hero', 'banner_image', url);

```

Como a página principal consome o CMS

```

// A página principal JÁ FAZ queries ao Supabase para buscar conteúdo

// O CMS apenas ATUALIZA esses dados — a home reflete automaticamente

// Exemplo de query na home (não alterar — já existe):

const { data: heroContent } = await supabase

    .from('site_content')

    .select('*')

    .eq('section', 'hero');

const { data: depoimentos } = await supabase

    .from('testimonials')

    .select('*')

```

```
.eq('active', true)
```

```
.order('order_index');
```

Permissões de acesso ao CMS

Nem todo perfil da secretaria deve acessar o CMS:

- **Admin** → acesso total ao CMS
- **Secretaria** → acesso apenas a cursos, alunos, turmas e financeiro
- **Instrutor** → sem acesso ao CMS

Implementar verificação de role antes de renderizar o menu "Conteúdo do Site".

Atualizado em Maio 2026 — AgênciaDeIA | Piticalyn